PROYECTO DE APLICACIONES MÓVILES · GUÍA DE EXPOSICIÓN

ServiNow

Qué hicimos, qué utilizamos y cómo explicarlo

Integrantes: Santiago Dorado, Juan David Saza y Fabián López.
Fecha: 11 de noviembre de 2026.

1. ¿Qué es ServiNow?

Es una aplicación Android que conecta a personas que necesitan un servicio con trabajadores que pueden realizarlo. El cliente publica su necesidad y los trabajadores presentan propuestas. Cuando se acepta una propuesta, ambas personas pueden coordinar el servicio mediante un chat privado.

Presentación inicial para decir en voz alta

“Nuestro proyecto se llama ServiNow. Lo desarrollamos para facilitar la búsqueda y contratación de servicios. Permite publicar trabajos, recibir propuestas, aceptar a un trabajador, conversar por chat y dar el servicio por finalizado. También incluye mapas, fotografías, avisos y una herramienta de inteligencia artificial para mejorar la descripción de las solicitudes.”

2. Recorrido principal de la aplicación

1. El usuario se registra o inicia sesión.

2. El cliente crea una publicación con descripción, ubicación y, si lo desea, una fotografía.

3. La publicación pasa por revisión antes de aparecer para la comunidad.

4. Los trabajadores consultan las publicaciones y presentan propuestas.

5. El cliente acepta una propuesta y se habilita la conversación privada.

6. Al terminar el servicio, el cliente puede marcar el trabajo como finalizado.

Además, la aplicación incluye perfiles, estadísticas, logros, avisos y funciones de moderación.

3. Herramientas para construir la app

Tecnología	Función en ServiNow
Android Studio y Kotlin	Entorno de desarrollo y lenguaje con el que programamos la aplicación.
Jetpack Compose	Construcción de pantallas, formularios, botones, tarjetas y listas.
Navigation Compose	Navegación entre las pantallas y organización de sus destinos.
ViewModel y StateFlow	Administración del estado de las pantallas y actualización de la interfaz cuando cambian los datos.
Repositorios	Separación de las operaciones de datos respecto a la lógica de las pantallas.
Hilt	Inyección de dependencias: proporciona los componentes que necesitan otras clases.
Coil	Carga y visualización de fotografías dentro de la interfaz.
Recursos de Android	Textos en español e inglés y selección del idioma.
Git y GitHub	Control de versiones y almacenamiento del código fuente con su historial.

4. ¿Cómo organizamos el código?

Utilizamos una organización por responsabilidades basada en MVVM y repositorios. La interfaz muestra la información; el ViewModel procesa las acciones y mantiene el estado; los repositorios leen o modifican los datos. Los modelos representan entidades como usuarios, publicaciones, propuestas y mensajes.

Flujo resumido: Pantalla → ViewModel → Repositorio → Servicio de datos. Los cambios de estado regresan a la interfaz para mostrar la información actualizada.

“Esta separación nos permite cambiar la forma de guardar datos sin tener que reescribir toda la pantalla. También facilita localizar errores y probar partes del proyecto.”

5. Servicios externos utilizados

Servicio	Para qué se utiliza
Firebase Authentication	Registro, inicio de sesión con correo o Google y recuperación mediante un enlace seguro enviado al correo.
Cloud Firestore	Persistencia y sincronización de publicaciones, propuestas, perfiles, conversaciones y avisos entre usuarios.
Cloudinary	Almacenamiento de fotografías en la nube. Firestore conserva los enlaces de las imágenes.
Mapbox	Visualización del mapa y selección de la ubicación del trabajo.
OpenRouter	Acceso al modelo de IA que ayuda a mejorar las descripciones de los trabajos.
Cloudflare Workers	Ejecución de operaciones del servidor y protección de claves privadas de los servicios externos.
Firebase Cloud Messaging	Envío de notificaciones al dispositivo por mensajes y cambios relacionados con los trabajos.
Firebase App Distribution	Distribución del APK release a personas invitadas para probar la aplicación.

Ejemplo: ¿qué sucede al enviar un mensaje?

“La pantalla comunica la acción al ViewModel. El repositorio guarda el mensaje en Firestore. La otra persona recibe el cambio en su conversación y el servidor solicita una notificación mediante Firebase Cloud Messaging.”

Dos aclaraciones importantes

Fotos: el archivo de la imagen se guarda en Cloudinary; en Firestore se guarda su URL. Así podemos recuperar y mostrar la fotografía desde distintos dispositivos.

Contraseñas: Firebase Authentication administra las credenciales. Para recuperarlas se envía un enlace de restablecimiento; no enviamos la contraseña actual por correo.

6. Mejoras y correcciones realizadas

Publicaciones y propuestas: Corregimos la consulta de información entre diferentes cuentas.

Fotografías: Arreglamos la carga a la nube y la visualización de imágenes en la aplicación.

Chat privado: Habilitamos la conversación entre el cliente y el trabajador después de aceptar una propuesta.

Inteligencia artificial: Mejoramos las instrucciones, las respuestas y el manejo de errores. La sugerencia se revisa antes de aplicarla al texto.

Notificaciones: Añadimos avisos internos, contador de pendientes y apertura de la pantalla relacionada al tocar un aviso.

7. ¿Cómo comprobamos el funcionamiento?

Se registraron **55 pruebas unitarias de Android** y **27 pruebas del servidor** aprobadas. También se realizaron pruebas con servicios reales para imágenes, chat y notificaciones, incluyendo la recepción de un aviso con la app en segundo plano y la apertura del chat.

En Firebase App Distribution aparecen dos pruebas de rastreo automatizadas superadas. Estos resultados corresponden a los recorridos ejecutados; no significan que todos los flujos posibles estén libres de errores ni sustituyen las pruebas manuales del equipo.

8. Uso de inteligencia artificial

Durante el desarrollo: usamos Codex como apoyo para implementar, revisar y corregir código. El equipo debe comprender los cambios, revisar lo generado y comprobar su funcionamiento.

Dentro de la aplicación: OpenRouter permite que el usuario mejore la redacción de una solicitud. La IA no debe inventar precios, causas del problema ni condiciones que el usuario no haya indicado.

“Son dos usos diferentes: una IA nos apoyó al desarrollar el proyecto y otra función de IA forma parte de la experiencia del usuario en ServiNow.”

9. ¿Qué hicimos para entregar el APK?

Generamos **ServiNow 1.1, código de versión 2**, como APK release. Esta es la versión preparada para distribuir, diferente de la versión debug utilizada durante el desarrollo.

1. Creamos una llave propia para firmar la aplicación.

2. Guardamos la llave y su contraseña fuera del repositorio.

3. Configuramos Gradle y activamos R8 para optimizar y reducir el código y los recursos no utilizados.

4. Registramos en Firebase la huella SHA-1 de la firma release para permitir el acceso con Google.

5. Generamos y verificamos el APK firmado.

6. Subimos el APK a Firebase App Distribution y agregamos el correo institucional como verificador.

Estado de la distribución: el correo juand.sazaa@uqvirtual.edu.co aparece como invitado. La aceptación y descarga dependen de que el usuario abra la invitación.

Explicación breve para la exposición

“No entregamos solamente el código. También generamos una versión instalable, firmada con una llave propia y optimizada con R8. La subimos a Firebase App Distribution para que los verificadores autorizados puedan descargarla y probarla.”

10. Preguntas que pueden hacerles

Pregunta	Respuesta breve
¿Dónde están los datos?	En Cloud Firestore. Las fotografías están en Cloudinary y Firestore guarda sus enlaces.
¿Qué hace la IA?	Ayuda a redactar mejor las descripciones; el usuario conserva la decisión de aplicar la sugerencia.
¿Por qué usar ViewModel?	Para separar el estado y la lógica de la pantalla respecto a su representación visual.
¿Por qué firmar el APK?	La firma identifica la aplicación y permite que Android compruebe la continuidad de las actualizaciones.
¿Para qué sirve App Distribution?	Para entregar versiones de prueba a personas invitadas, sin publicarlas en Google Play.

11. Guion de cierre

“En conclusión, ServiNow integra el flujo de publicación, propuestas y coordinación de servicios con herramientas reales en la nube. Utilizamos Firebase para autenticación y datos, Cloudinary para imágenes, Mapbox para mapas y OpenRouter para la ayuda de redacción. Organizamos el código con ViewModel y repositorios, realizamos pruebas y distribuimos un APK release mediante Firebase App Distribution.”

Sugerencia para repartir la exposición

Integrante 1: objetivo de ServiNow y demostración del recorrido cliente-trabajador.

Integrante 2: arquitectura, tecnologías y servicios externos.

Integrante 3: mejoras, pruebas, uso de IA y distribución del APK.

Esta distribución es una sugerencia; los tres integrantes deben poder explicar el funcionamiento general y las decisiones del proyecto.

Demostración recomendada

Preparar dos cuentas de prueba. Mostrar una publicación, una propuesta aceptada, el chat y un aviso. Después enseñar el selector de idioma, una fotografía y la ayuda de IA. Cerrar mostrando la versión 1.1 (2) en App Distribution.

Antes de exponer, comprobar la conexión a internet y el acceso a las cuentas. Evitar mostrar contraseñas, claves privadas o datos personales de otros usuarios.

Referencias y evidencia del proyecto

Guía de generación del APK:
caflorezvi.github.io/guias-apps-moviles/21.generar-apk.html

Repositorio del proyecto:
github.com/JDFaFa/ServiNow

Firebase App Distribution:
Abrir versiones de ServiNow en Firebase (requiere acceso autorizado).

Esta guía resume el trabajo realizado y los resultados registrados en el proyecto. Sirve como apoyo para la exposición; no reemplaza la comprensión del código ni una revisión completa de la aplicación.